WEB DEVELOPMENT WITH

MVC Framework

Lecturer – Anupama Dilshan
Department of IT

Available popular PHP MVC frameworks

- **Laravel**
- Code ignitor
- Cake PHP
- Symphony
- ZEND

Why *Laravel* ?

- We will outline some of the interesting features of the Laravel framework here, which will explain why it is gaining so much popularity.

Authorization Technique:

- Laravel framework makes implementation of authentication techniques very simple. Almost everything is configured perfectly. Laravel provides a very simple way to organize authorization logic and control access to various resources.

Object-Oriented Libraries:

- Laravel is the best PHP framework as it has Object Oriented libraries and other pre-installed ones, which are not found in any other PHP frameworks. One of the pre-installed libraries is the Authentication library. It has many advanced features like checking active users, Bcrypt hashing, password reset, CSRF (Cross-site Request Forgery) protection, and encryption.

Artisan:

- Laravel framework offers a build in a tool named Artisan. This tool allows a developer to perform the majority of those repetitive programming tasks which most of the developers avoid performing manually.

```
make:channel          Create a new channel class
make:command          Create a new Artisan command
make:controller       Create a new controller class
make:event            Create a new event class
make:exception        Create a new custom exception class
make:factory          Create a new model factory
make:job              Create a new job class
make:listener         Create a new event listener class
make:mail             Create a new email class
make:middleware       Create a new middleware class
make:migration        Create a new migration file
make:model            Create a new Eloquent model class
make:notification     Create a new notification class
make:observer         Create a new observer class
make:policy           Create a new policy class
make:provider         Create a new service provider class
make:request          Create a new form request class
make:resource         Create a new resource
make:rule             Create a new validation rule
make:seeder           Create a new seeder class
make:test             Create a new test class
```

MVC Support:

- Another reason which makes Laravel the best PHP framework is it supports MVC Architecture like Symfony, ensuring clarity between logic and presentation. MVC helps in improving the performance, allows better documentation, and has multiple built-in functionalities. Here's how the MVC works for Laravel.

Security:

- Laravel takes care of the security within its framework. It uses the hashed password, which means that the password would never save as the plain text in the database. Laravel framework uses prepared SQL statements which make injection attacks unimaginable.

Database Migration

- With Laravel framework database migrations, it is extremely easy. As long as you keep all of the database work in migrations and seeds, you can easily migrate the changes into any other development machine you have.

Blade Template Engine:

- The Blade templating engine of the Laravel framework is very intuitive and helps to work with the typical PHP/HTML spaghetti so perfect, that’s it one of the best features of the Laravel framework.

Pre-Requisites before start Laravel

- Good Knowledge of PHP and basic Knowledge of PHP OOP concepts
- Good Knowledge in Bootstrap
- Good Knowledge in JavaScript and JQuery
- Good Knowledge in Web Technology
- Good Knowledge in SQL Database

Some Important Links for Reference

- https://www.w3schools.com/php/php_intro.asp
- https://getbootstrap.com/docs/4.0/getting-started/introduction/
- https://www.w3schools.com/jquery/jquery_intro.asp

Technical Requirements

- You Need Composer to install Laravel.

What is Composer ?

A composer is a tool, incorporates all the dominions and libraries. It enables a user to generate a project concerning the specified framework (for instance, those adopted in Laravel installation). Third party libraries can be installed efficiently with the help of composer.

- You need to have Database. You can use any supported database technology for Laravel

  Example : SQL , POSGRATE , MONGODB, SQL LITE

- You need to have some code editor (IDE)

  Example : VS CODE, ATOM, SUBLINE, BRACKETS

How to install Laravel 9 without UI & Authentication module

- Laravel is very simple to install
- All you have to do is install the PHP dependency management system (Composer)
- After that using composer we can install Laravel
- Create a folder any where in your computer Example :- C partition or D partition
- After that open up your command line editor (CMD)
- Navigate to above created folder though your CMD

CMD Navigation

- Before CMD navigation we need to understand some useful CMD commands

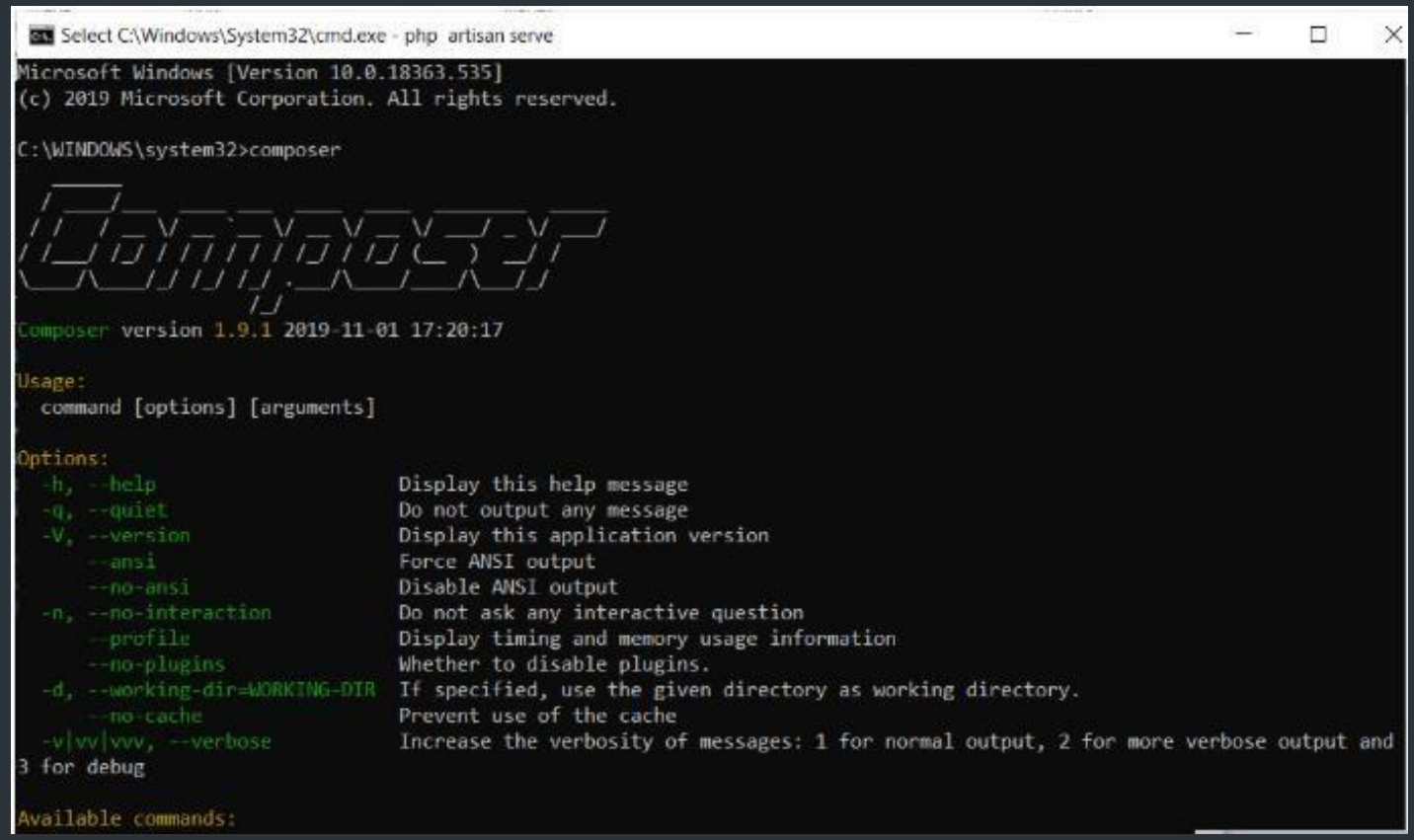
1. cd => Change directory
2. cd .. => Come out from the directory (Only one level out)
3. cls => Clear the Screen (Remove unwanted things)
4. mkdir => Make a directory (Create a new folder)

If you interest to find more CMD commands please follow this URL

https://helpdeskgeek.com/help-desk/21-cmd-commands-all-windows-users-should-know/

Install Laravel via Composer

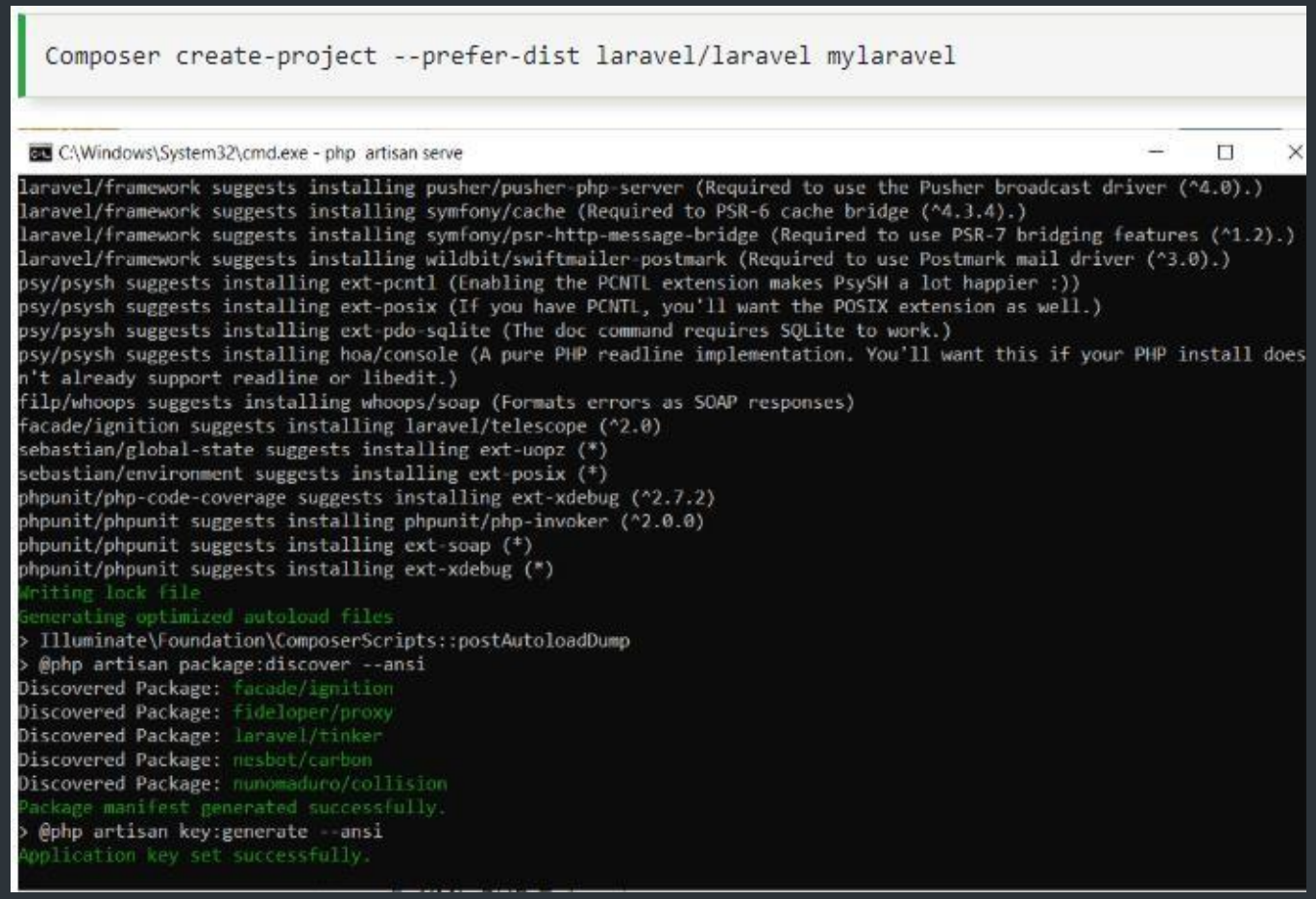
- **Check Composer Installed successfully type "composer" in command line**



After navigate to your working folder …

- Now you need to type the following command to create new Laravel project using composer.

```
Composer create-project --prefer-dist laravel/laravel mylaravel
```



Start your Server Now

- Laravel comes with inbuilt development server for development purpose we can start up the laravel development server by using following command

1. **php artisan serve**

   Now your Development server is UP and RUNING on port 8000

```
E:\xampp\htdocs\bsc_17\advance web\laravel\batch17>php artisan serve
Laravel development server started: http://127.0.0.1:8000
```

Install laravel with UI kit and Authentication Modules

- After the laravel installation run the following commands one by one.

1. composer require laravel/ui –dev
2. php artisan ui vue –auth

After that you need to run some npm command, before that you need to download and install node.js after that you can execute the any required npm command

node.js => https://nodejs.org/en/

laravel authentication => https://laravel.com/docs/6.x/authentication

- If you want to start your development server on different port, you can try following artisan command

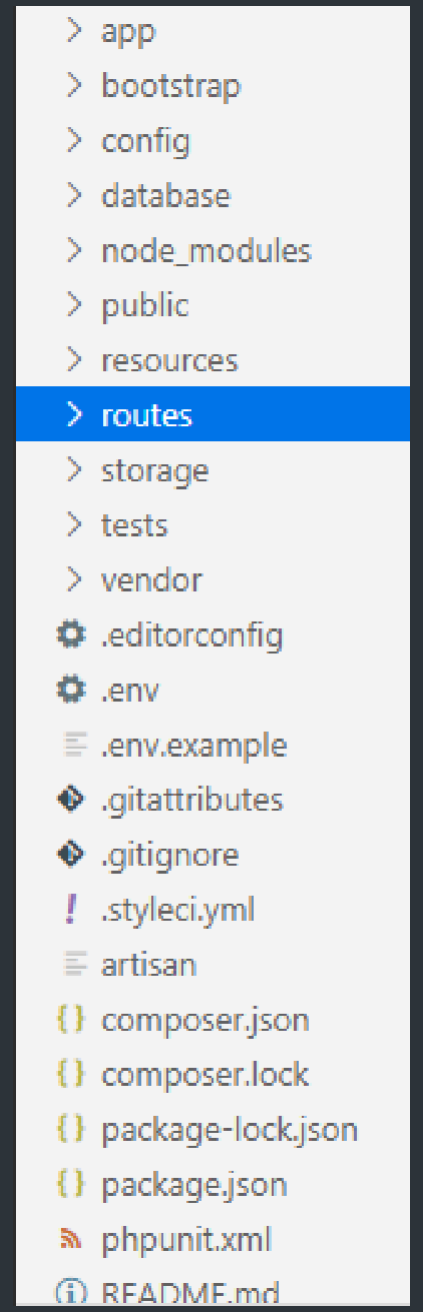
  php artisan serve –port=8012

```
E:\xampp\htdocs\bsc_17\advance web\laravel\batch17>php artisan serve --port=80
12
Laravel development server started: http://127.0.0.1:8012
```

Laravel Folder Structure

- Actually you don't want to memorize each and every folder or files in laravel, but generally you need to have good understanding of following folders and files, when its comes to the development it's more easy to navigate between files and folders if you have good understanding in basic structure of Laravel

1. app
2. database
3. public
4. resources
5. routes
6. test
7. .env



MVC MODULES in Laravel

- Lets talk about MVC modules in laravel, In here you will get clear understand about Routers | Controllers and Views modules. If you don’t have any idea about MVC architecture please go back to my MVC architecture slides.

What Is a Route?

- Route is a way of creating a **request URL** of your application. These URL do not have to map to specific files on a website. The best thing about these URL is that they are both human readable and SEO friendly.
- In Laravel 9, routes are created inside the **routes** folder. Routes for the **website** are created in **web.php** file, similary the routes for **API** are created in **api.php**

Routing in Laravel includes the following categories :

- Basic Routing
- Route parameters
- Named Routes

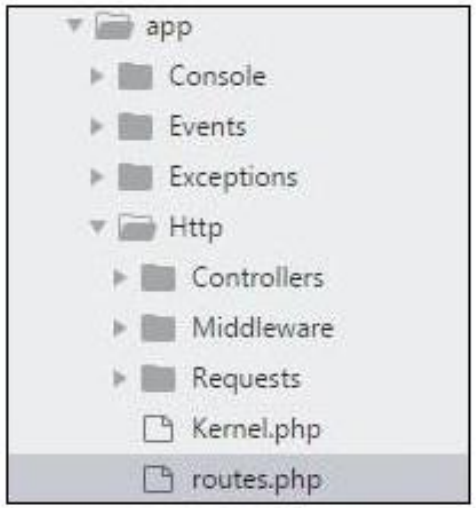
- In the above code, a route is defined for the homepage. Every time this route receives a get request for /, It will return the view **welcome.** Views are the frontend of the application and I will discuss the topic in an upcoming installment of this series.

- The structure of route is very simple. Open the appropriate file (either `web.php` or `api.php`) and start the code with `**Route::**` This is followed by the request you want to assign to that specific route and then comes the function that will be executed as a result of the request.

Laravel offers the following route methods:

- get
- post
- put
- delete
- patch
- options

Basic Router




```
Route::get ('/', function () {
   return view('welcome');});
```

Example

Observe the following example to understand more about Routing −

app/Http/routes.php

```
<?php
Route::get('/', function () {
   return view('welcome');
});
```

Redirect Routes

If you are defining a route that redirects to another URI, you may use the **Route::redirect** method. This method provides a convenient shortcut so that you do not have to define a full route or controller for performing a simple redirect:

```
Route::redirect('/here', '/there');
```

View Routes

If your route only needs to return a view, you may use the Route::view method. Like the redirect method, this method provides a simple shortcut so that you do not have to define a full route or controller. The view method accepts a URI as its first argument and a view name as its second argument. In addition, you may provide an array of data to pass to the view as an optional third argument:

```
Route::view('/welcome', 'welcome');

Route::view('/welcome', 'welcome', ['name' => 'Taylor']);
```

Route Parameters

- Sometimes in the web application, you may need to capture the parameters passed with the URL. For this, you should modify the code in **routes.php** file.
- You can capture the parameters in **routes.php** file in two ways

1. Required Parameters
2. Optional Parameters

Required Parameters

- These parameters are those which should be mandatorily captured for routing the web application. For example, it is important to capture the user's identification number from the URL. This can be possible by defining route parameters as shown below :-

```
Route::get('ID/{id}',function($id) {
   echo 'ID: '.$id;
});
```

- You may define as many route parameters as required by your route:

```
Route::get('posts/{post}/comments/{comment}', function ($postId, $commentId) {
    //
});
```

Route parameters are always encased within `{}` braces and should consist of alphabetic characters, and may not contain a `-` character. Instead of using the `-` character, use an underscore (`_`). Route parameters are injected into route callbacks / controllers based on their order - the names of the callback / controller arguments do not matter.

Optional Parameters

- Sometimes developers can produce parameters as optional and it is possible with the inclusion of **?** after the parameter name in URL. It is important to keep the default value mentioned as a parameter name. Look at the following example that shows how to define an optional parameter:-

```
Route::get('user/{name?}', function ($name = null) {
    return $name;
});

Route::get('user/{name?}', function ($name = 'John') {
    return $name;
});
```

Named Routes

- Named routes allow a convenient way of creating routes. The chaining of routes can be specified using name method onto the route definition. The following code shows an example for creating named routes with controller

```
Route::get('user/profile', 'UserController@showProfile')->name('profile');
```

- The user controller will call for the function **showProfile** with parameter as **profile**. The parameters use **name** method onto the route definition.

Laravel Controller

- Developer can create business logic of the entire application using Controllers.
- Always Controllers and application logics are separated
- Application view and Controllers are link using routes.
- We can categorize Controllers in to major three categories
  1. Basic Controller
  2. Middleware Controller
  3. Resource Controller

Basic Controller

- Creating a Controller

  Open up your CMD and type the following command

```
php artisan make:controller <controller-name>
```

  The controller that you have created can be invoked from within routes.php file using this syntax below

```
Route::get('base URI','controller@method');
```

Example code for basic controller

```php
<?php

namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Http\Requests;
use App\Http\Controllers\Controller;

class AdminController extends Controller
 {
 //
 }
```

Controller Middleware

- You can assign controllers to middleware's to route in the route files of your project using the command below

```
Route::get('profile', 'AdminController@show')->middleware('auth');
```

- Middleware methods from the controller help to easily assign middleware to the controller's action and activity. Restrictions on implementing certain methods can also be provided to middleware's on the controller class.

```
class AdminController extends Controller
{
    public function __construct()
    {
        // function body
    }
}
```

What is a Middleware ?

- Middleware provide a convenient mechanism for filtering HTTP requests entering your application. For example, Laravel includes a middleware that verifies the user of your application is authenticated. If the user is not authenticated, the middleware will redirect the user to the login screen. However, if the user is authenticated, the middleware will allow the request to proceed further into the application.

- Additional middleware can be written to perform a variety of tasks besides authentication. A CORS middleware might be responsible for adding the proper headers to all responses leaving your application. A logging middleware might log all incoming requests to your application

- There are several middleware included in the Laravel framework, including middleware for authentication and CSRF protection. All of these middleware are located in the **app\Http\Middleware** directory

- To create a new middleware, use the make::middleware Artisan command

```
php artisan make:middleware CheckAge
```

- This command will place a new checkAge class within our app\Http\Middleware directory. In this middleware we will only allow access to the route is the supplied age is greater than 200. otherwise we will redirect the user back to the home page or login page.

```php
<?php

namespace App\Http\Middleware;

use Closure;

class CheckAge
{
    /**
     * Handle an incoming request.
     *
     * @param  \Illuminate\Http\Request  $request
     * @param  \Closure  $next
     * @return mixed
     */
    public function handle($request, Closure $next)
    {
        if ($request->age <= 200) {
            return redirect('home');
        }

        return $next($request);
    }
}
```

Resource Controllers

- The resource route of Laravel allows the classic "CRUD" routes for controllers having a single line of code. This can be created quickly using the make: controller command (Artisan command) something like this“

```
php artisan make:controller PasswordController --resource
```

Action handled by Resource controllers

Verb	URI	Action	Route Name
GET	/users	Users list	users.index
POST	/users/add	Add a new user	users.add
GET	/users/{user}	Get user	users.show
GET	/users/{user}/edit	Edit user	users.edit
PUT	/users/{user}	Update user	users.update
DELETE	/users/{user}	Delete user	users.destroy

Laravel Model

Introduction

The Eloquent ORM included with Laravel provides a beautiful, simple Active Record implementation for working with your database. Each database table has a corresponding "Model" which is used to interact with that table. Models allow you to query for data in your tables, as well as insert new records into the table.

Before getting started, be sure to configure a database connection.

Defining Model

- To get started with Eloquent, we have to create a model first
- Models are typically live in **app** directory , The easiest way to create a model instance is using the **make:model User** (Make sure Model name is similar to its table migration singular and first letter is always capital)
- If you like to generate a database migration when you generate the Model, you may use –migration or –m option

```
php artisan make:model Flight --migration

php artisan make:model Flight -m
```

Laravel model

ClassName	Singular of table name	`protected $table = 'custom_name'`
Primary key	id	`protected $primaryKey`
Timestamp	created_at, updated_at	`protected $timestamp = false`
Guarded	array of fields name	`protected $guarded = array('id', 'password')`
Fillable	array of fields name	`protected $fillable = array('id', 'password')`

Eloquent Methods

- Please reoffer the following URL

  https://laravel.com/docs/5.8/eloquent

Class Project …

- Create a E-channeling Application, your application must include the following requirements

1. Patient login
2. Patient can make a appointment via channeling form
3. Application must contain admin login
4. Admin have privileges to view patient appointments
5. Admin have privileges to delete or confirm the patient booking

Thank You